

CB-Manager and LCP: enhance the Security Cloud with the Programmability.

Authors

Alessandro Carrega - alessandro.carrega@cnit.it - CNIT

Matteo Repetto - matteo.repetto@ge.imati.cnr.it - CNR

Abstract

ASTRID is a multi-tier architecture, where a common, programmable, and pervasive context fabric feeds a powerful set of multi-vendor detection and analysis algorithms (business logic). On the one hand, the challenge is deep visibility over multiple software components by real-time collection of massive events from a multiplicity of capillary sources, while maintaining essential properties such as forwarding speed, scalability, autonomy, usability, fault tolerance, resistance to compromises, and responsiveness. On the other hand, the ambition is to support better and more reliable situational awareness by inter- and intra-domain data correlation in both space and time, in order to timely detect and respond even the more sophisticated multi-vector and interdisciplinary cyberattacks.

The Context Broker (CB) is the logical component to manage the security context. We define the security context as the set of information, data, and measurements that describe the service and can be used for security-related purposes. The scope includes the description of the service components (namely, software deployed in each virtual function) and its topology (network links and communication channels), as well as operational data from the execution of the service (logs, system metrics, measurements, events, software traces).

The CB-Manager function implements a REST API that provides a uniform access interface to multiple components and the information stored in the CB's database: (i) detection and monitoring agents deployed in virtual functions: location, capability, properties; (ii) external service orchestrator: service topology, run-time parameters; (iii) security services: capability, parameters; and (iv) data stored in the CB, as reported by the running agents: retrieval, queries.

The Local Control Plane (LCP) gives the CB access to the configuration of agents. It is designed to support multiple configuration methods (through configuration file, command line, REST API) but to remain unaware of specific protocols. The LCP acts as the single point of contact for the CB and exposes the list of available agents.

The architecture includes a number of components to be deployed in the execution environment of Virtual Functions (VFs), and the internal structure of the CB. All components are organized in a data plane and a control plane; the management plane is not shown in the picture because it is entirely implemented by the external software orchestration tool. The overall design is based on

the Elastic Stack framework¹, a collection of open source projects for data acquisition, processing, and storage. It was originally composed of Elasticsearch, Logstash, and Kibana (for which it was formerly known as ELK) and is now evolving to include additional components. Though these tools were originally conceived to work statically, i.e. with minimal or no possibility to change the configuration at run-time, we are implementing an additional dimension of programmability in the control plane, so to easily support the definition of new inspection and monitoring tasks at run-time.

In the last part, we performed an evaluation of the CB-Manager and LCP considering different scenarios and workloads. The goal is to verify the robustness and the reliance of these two components in different execution scenarios.

1. Context Broker

The CB is the logical component to manage the security context. We define the security context as:

the set of information, data, and measurements that describe the service and can be used for security-related purposes. The scope includes the description of the service components (namely, software deployed in each virtual function) and its topology (network links and communication channels), as well as operational data from the execution of the service (logs, system metrics, measurements, events, software traces).

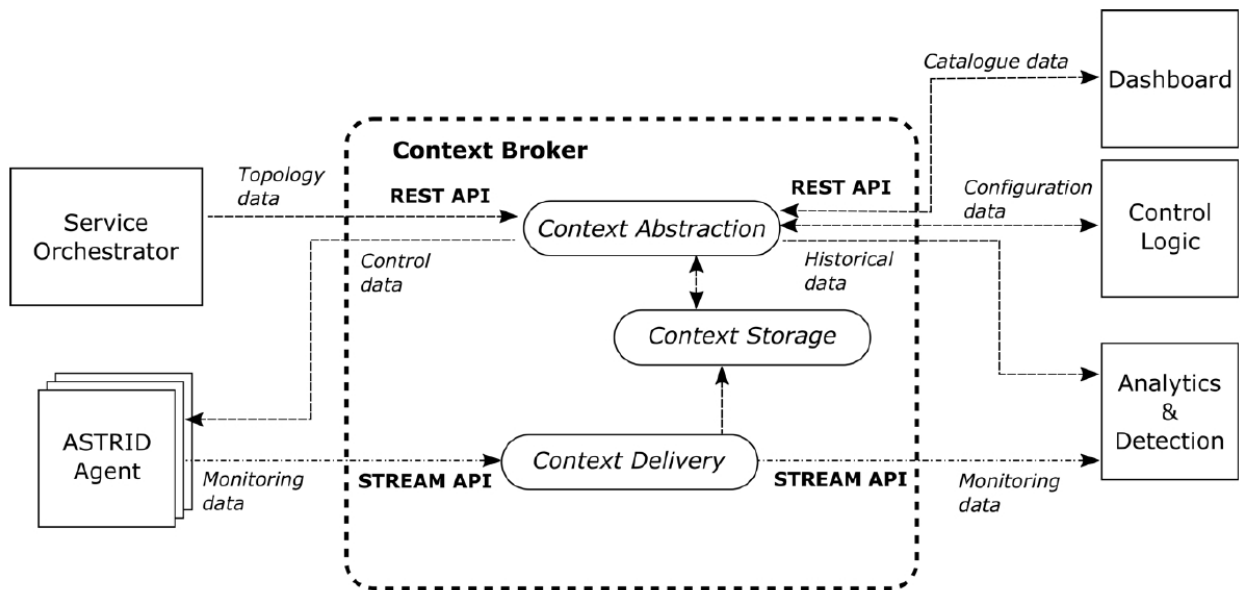


Figure 1. Conceptual model of the CB.

Conceptually, the CB implements three main logical functions:

Context Delivery, which is the real-time collection of data and measurements generated by local agents according to streaming patterns. The internal delivery function is expected to make this data directly available to intended consumers (i.e., analytics and detection algorithms implemented over the ASTRID platform) and to store it internally.

¹ The Elastic Stack. URL: <https://www.elastic.co/elk-stack>.

Context Storage, which keeps historical data for offline analysis. It includes the whole context, therefore encompassing both service topology, available agents, and data generated by them.

Context Abstraction describes the overall service topology, including available agents, their capabilities and their current configuration. This information is used by the control logic to configure agents, so to collect data that are needed by specific algorithms. Analytics and detection algorithms can also use this information for correlating data and measurements based on the service topology, as well as to retrieve and make selective queries on historical data.

The first task for the CB is to manage the heterogeneity of sources and protocols, which is reflected in different data and control interfaces. The CB hides this heterogeneity and exposes a common context model to the other components in the security orchestrator through the Context Abstraction function, for discovering, configuring, and accessing the security context available from the execution environment. Though control and management of local agents is a common feature in existing SIEM tools, automatic discovery of the service topology is currently not available, but this is very useful for cloud-based services, where the composition and topology are expected to autonomously change during the lifetime according to the evolving context.

The CB collects data from monitoring and inspection processes deployed in the execution environment (Context Delivery and Context Storage), hence implementing the required data channel envisioned by the ASTRID architecture in Figure 1. The CB hides the heterogeneity and asynchrony of the sources, feeds the analysis algorithms with the requested context, organizes historical data, and provides simple querying and fusion capabilities in data access. Given the very different semantics of the context data, the obvious choice is non-relation databases (NoSQL). This allows defining different records for different sources, but also poses the challenge to identify a limited set of formats, otherwise part of the data might not be usable by some algorithms.

The flexibility in programming the execution environment is expected to potentially lead to a large heterogeneity in the kind and verbosity of data collected. For example, some virtual functions may report detailed packet statistics (i.e., those at the external boundary of the service), whereas other functions might only report application logs. In addition, the frequency and granularity of reporting may differ for each virtual function. The definition of a (security) context model is therefore necessary for detection algorithms to know what could be retrieved (i.e., capabilities) and what is currently available, how often, with each granularity (i.e., configuration). The implementation of this model is the Security Context Abstraction, the homogeneous control interface that the CB offers for configuring and programming different data sources, by implementing the specific protocols, corresponding to the control channel in Figure 1. This is intended to be used during the design of analytics pipelines, to select and configure agents so that they produce data according to the format and content expected by the analytics and detection algorithms.

Finally, Table 1 maps the logical functions of the CB to the software architecture.

Table 1. Mapping of CB functions to the software architecture.

Function	Software components
Software components	CB-Manager
Context Abstraction	Kafka

2. CB-Manager

The CB-Manager function implements a REST API that provides a uniform access interface to multiple components and the information stored in the CB's database:

- detection and monitoring agents deployed in virtual functions: location, capability, properties;
- external service orchestrator: service topology, run-time parameters;
- security services: capability, parameters;
- data stored in the CB, as reported by the running agents: retrieval, queries.

The CB-Manager interface uses a single protocol but exposes different data models for each component, due to the large heterogeneity in the configuration properties of similar tools. The backend database for the CB-Manager is Elasticsearch. Different indexes are used for data produced by agents, notifications sent by the security services, and context information. Context information includes the data models for agents, security services, and virtual services.

The main indexes used are types, catalogs, instances and data. Types and catalogs are managed by the system administrator to describe the objects available within the system: execution environments (virtual machines, containers), network links (LAN, point-to-point, point-to-multipoint), agents, algorithms, and programs. Instances describe concrete instantiations of the objects, including virtual functions, agents, security services. They are typically created and managed by the Security Controller, based on the events reported by the external software orchestrator. Data is generated by agents and can only be inserted through the data channel; the REST API only supports retrieval and queries on the data. This is mainly provided to hide the backend storage to the dashboard.

Interfaces. The CB-Manager implements the following interfaces (Figure 2):

- REST APIs for interacting with the internal data models (Eastbound), consumed by the Security Controller and the Dashboard;
- consumer of the REST APIs for interacting with the LCP (Westbound);
- consumer of Elasticsearch API for querying and retrieving objects in the datastore (Southbound).

Supported features. The current implementation supports the following features:

- creation, modification, deletion, and retrieval of object types and instances;
- creation, modification, deletion, and retrieval of objects in the catalog;
- pushing agent configurations/programs to the LCP when the corresponding data model is updated;
- authorization and access control on the Eastbound interface (HTTP and JWT methods available).

Ancillary components. The CB-Manager needs data models for

- object types;

- agents;
- programs that can be injected into specific agents (e.g., eBPF programs for DynMon, filters for Logstash);
- security services.

Data models are partially available, primary for the demos realized so far.

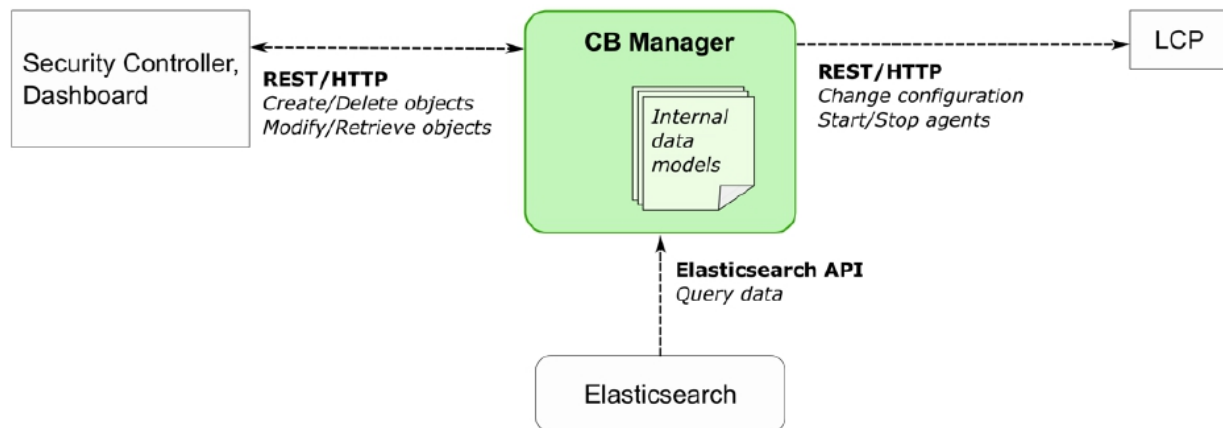


Figure 2. The CB-Manager and its interfaces.

3. Local Control Plane (LCP)

The LCP gives the CB access to the configuration of agents. It is designed to support multiple configuration methods (through configuration file, command line, REST API) but to remain unaware of specific protocols. The LCP acts as the single point of contact for the CB and exposes the list of available agents.

Standard beats from the Elastic stack (Filebeat, Metricbeat, Packetbeat) are configured by changing their configuration files, since there are no available APIs to dynamically program these agents. They can change predefined options in each configuration file, according to the programming model exposed by the CB and restart the beat to apply the new configuration through the LCP. Indeed, the LCP is also used to invoke shell commands and scripts, which are used for starting/stopping the beat, so that no data is generated if not requested by any consumer in the framework. Cubes developed in the Polycube framework are directly configured through their REST API, which is exposed by each cube. In this case, the LCP is mostly a proxy that forwards HTTP messages to the Polycube daemon. There is no need for shell commands in this case, since start/stop operations of cubes are directly managed by polycubed.

The LCP plays the role of local control point to give access to agents' configurations. It does not provide any monitoring data or event itself, but it is only used for configuring a heterogeneous set of agents. The LCP is deployed by the external service orchestrator, as any other local agent, and its configuration must include the list of installed agents. The LCP supports multiple configuration and control methods:

- editing of YAML files;
- forwarding of HTTP messages to the agent interface;

- invoking shell commands (mainly to start/stop/reload the agents).

The LCP can be defined as a dump configuration entity, which is used to get remote access to local agents. Indeed, it is not aware of the semantics of configuration messages: it just applies what the CB Manager provides. That means no drivers or plugins are required for the agents; just the knowledge of what configuration method should be used. The LCP allows remote control even for those agents that do not provide a management interface. There is no need to open multiple ports on the firewall, and authentication and access control can be applied to a single entry point instead of many different interfaces. The advantage of implementing a LCP instead of relying on more general-purpose mechanisms (e.g., SSH connection) are the following:

- there is a consolidated practice for using implementing REST APIs on top of HTTP, and integration with data models is rather straightforward;
- parsing HTTP messages and applying access control is simpler than with a plain SSH connection.
- the local configuration agent might be used to apply default and fallback configurations in case the centralized platform is not reachable.

Once activated, the LCP is able to respond to connection requests from the CB-Manager. It can interact with different CB Manager instances at the same time. The CB-Manager sends periodic heartbeating messages to verify the connectivity to the LCP. The heartbeating messages are also used to periodically renew the security credentials. The LCP then uses the most appropriate method to apply incoming configuration requests. The LCP is explicitly designed to interact with the CB-Manager, and no other kinds of usage are expected; hence, the REST requests must be made only from the CB-Manager.

However, for development and test purposes, common tools for testing REST interfaces can be used. Access control is delegated to the IAM, but this integration is still on-going.

Interfaces. The LCP implements the following interfaces (Figure 3):

- REST APIs for control of local agents (Eastbound), consumed by the CB Manager,
- file system access to edit files (Southbound);
- shell command execution to invoke management actions on local agents (Southbound);
- HTTP client to forward messages to the interfaces of local agents (Southbound).

Supported features. The current implementation supports the following features:

- editing of configuration files written with YAML syntax;
- forward of configuration messages to a local Polycube instance;
- execution of any command on a local shell;
- authentication on the Eastbound interface.

Ancillary components: None. No specific components are needed for operation of the LCP.

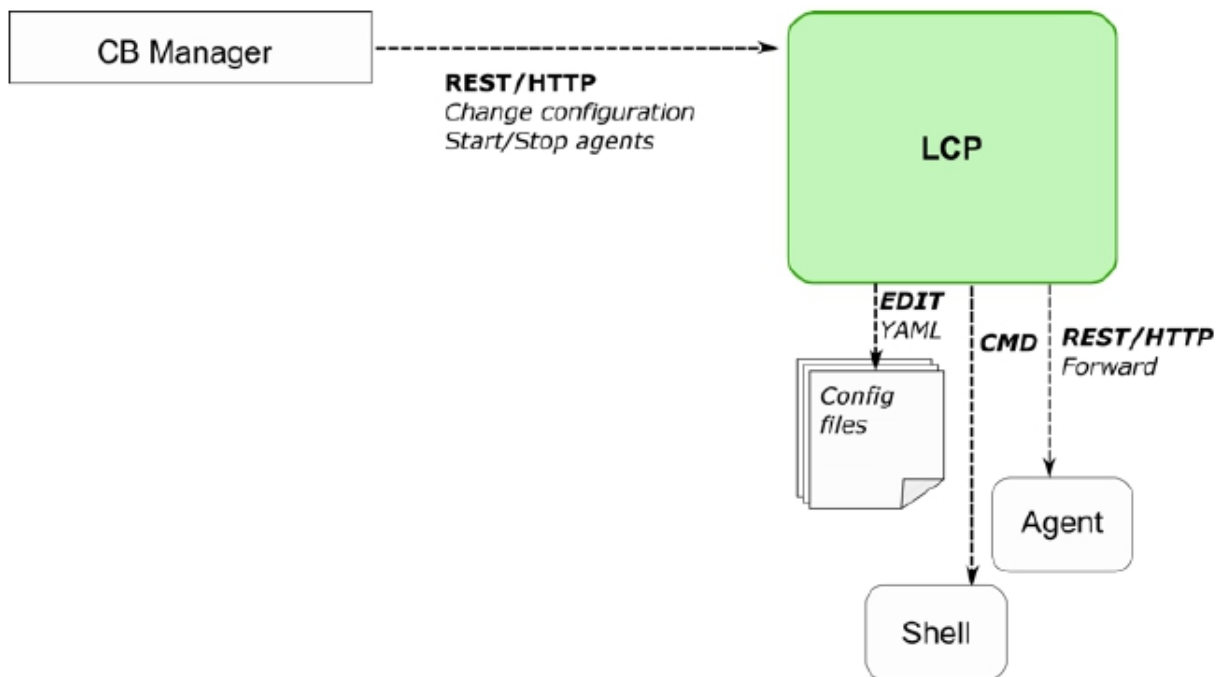


Figure 3. The Local Control Plane (LCP) and its interfaces.

4. Software Architecture

Figure 4 shows the software architecture of the context framework, based on the conceptual model depicted in Figure 1. The architecture includes a number of components to be deployed in the execution environment of Virtual Functions (VFs), and the internal structure of the CB. All components are organized in a data plane and a control plane; the management plane is not shown in the picture because it is entirely implemented by the external software orchestration tool. The overall design is based on the Elastic Stack framework², a collection of open source projects for data acquisition, processing, and storage. It was originally composed of Elasticsearch, Logstash, and Kibana (for which it was formerly known as ELK) and is now evolving to include additional components. Though these tools were originally conceived to work statically, i.e. with minimal or no possibility to change the configuration at run-time, we are implementing an additional dimension of programmability in the control plane, so to easily support the definition of new inspection and monitoring tasks at run-time, according to the overall programming model for the framework.

² The Elastic Stack. URL: <https://www.elastic.co/elk-stack>.

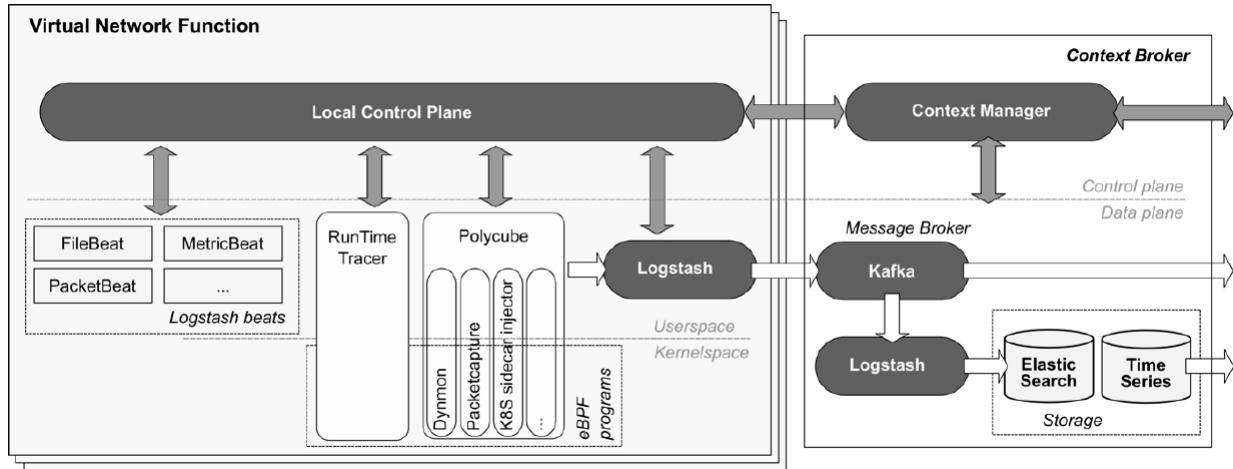


Figure 4. Software architecture of the context fabric.

In the execution environment of the Virtual Function (in short ExecEnv), the architecture provides a number of complementary agents for monitoring, inspection, tracing, and enforcement. The design of the data plane revolves around the Logstash component. The data plane includes Elastic Beats, which are monitoring agents conceived as specific extensions of Logstash for parsing files, inspecting packets, and gathering system information. In addition to standard Beats, some other monitoring components are being developed that collect more measurements and data from the ExecEnv and that inspect packets and system calls, as required by the current definition of the SCM. The above additional tools will also feed Logstash, which will implement data enrichment before sending the data to the CB. Logstash will also act as buffer, being able to temporarily store data in case of intermittent connectivity (indeed, the same function is also present in Kafka). The monitoring and inspection processes will be implemented both in userspace (e.g., common monitoring tools and interfaces already available in the operating system) and in the kernel (by using eBPF - extended Berkeley Packet Filters - programs). A specific agent is the Polycube framework. It offers a flexible API (Application Program Interface) for managing multiple processes (denoted as cubes); this interface includes both general control actions and specific operations to be forwarded to each cube. Several cubes are being developed for specific functions, mostly concerning packet filtering and injection of eBPF programs in the tc hook.

The control plane is implemented by the Local Control Plane (LCP). It acts like a sort of configuration proxy that allows the Context Manager to access local agents. The LCP supports the most common methods for configuring and running agents: it can change configuration files, run shell commands, forward HTTP (HyperText Transfer Protocol) messages to local ports. However, it is unaware of the specific protocols and formats³; in this way, no modifications are necessary when new agents are added.

The control plane of the CB includes a message broker to collect data from the ExecEnv and deliver it to the intended recipients; it is implemented by Kafka, a tool which is already

³ For instance, in case of configuration by file, the LCP receives the configuration file, its name, and location within the file system by the Context Manager. The LCP save the configuration file in that location but does not know anything about its content.

wellintegrated in the Elastic Stack. A special recipient, present within the CB, is the storage component, which is responsible for conserving historical data in a structured way. An additional instance of Logstash is also envisioned in the CB for indexing data which are delivered to the storage module. This may also be useful for timestamping or other fingerprinting operation that might be required to give legal validity for forensics investigation; the specific pipeline will be set by each use case, according to specific requirements. Within the storage block, an Elasticsearch instance stores the context for off-line analysis (including visualization to humans). The time series DB is provided to efficiently store huge amount of data for forensics and legal investigation.

The Context Manager implements the control interface to the CB. Such interface exposes the SCM through the SCA and allows external entities to control local monitoring agents and to access historical data. The abstraction of the service topology, local agents, and security properties is based on a graph-like model, as common practice in many recent management protocols for software-defined networking.

The security context retrieved by the CB contains monitoring data that may disclose private and sensitive data. Access to this data should therefore be limited to authorized roles and elements, while confidentiality and integrity mechanisms should be used when transferring the data. In addition, configuration of the remote data plane must remain a prerogative of the security controller and trusted policies, so it is important to track the issuer of such commands. The CB is therefore expected to enforce access policies settled by the IDM module.

Security of the whole framework is mainly based on two aspects, targeting encryption and integrity mechanisms for both data at move and data at rest, whereas for data in use there are no specific security requirements. Encryption and integrity will be used in Kafka, to avoid transmitting plaintext information over unsecure network connections. Access to Kafka will be subject to preliminary authentication, with a different identity for every software component (detection algorithm, security controller, CB's Logstash instance); this will allow fine-grained access control to data. The same mechanisms will be used when transferring data from the internal Elasticsearch instance. For data at rest, Transparent Data Encryption (TDE) will be used to encrypt the whole database. Both control interfaces (e.g., exposed by Polycube and the CB) are based on REST messages, so protection will be based on the TLS layer present in the HTTPS protocol. Authentication and authorization will define ACLs for every tree or subtree of the data model.

architecture is conceived to implement analytics pipelines that are defined at run-time. The creation of an analytic pipelines involves a mix of configuration and programming, which allows both the reusage of existing components as well as the definition of new ones. Besides the architectural components, the overall programming model is depicted in Figure 5, with mapping to components. It is worth noting that the components described in this document only implement some parts of the overall programming model, namely those related to Raw data extraction, Data enrichment, and Indexing. The remaining parts (Stream processing, Batch processing, and Reaction) are covered by analytics algorithms and security orchestration. An overarching overview is anyway given in this document to provide the whole picture.

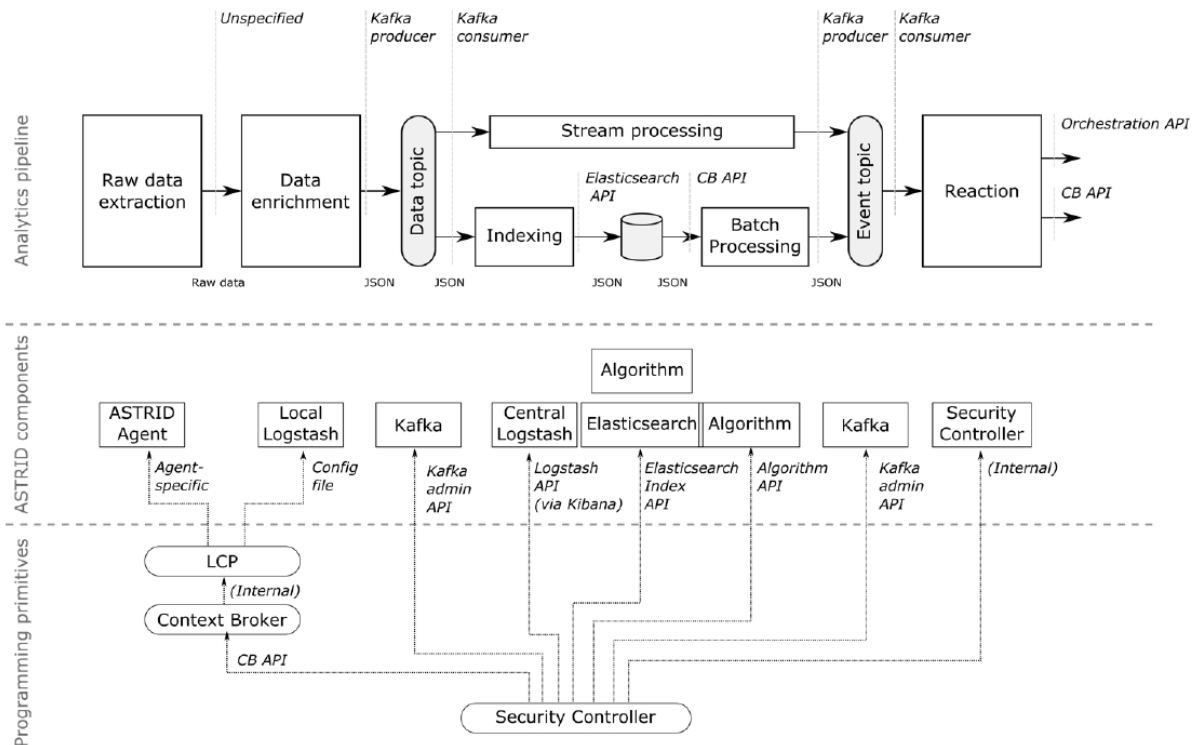


Figure 5. Programming model for an analytics pipeline and mapping to components.

An analytics pipeline is made of multiple programmable stages, which follow a common structure but can be tailored to the needs of different security services. The first stage extracts and collects data from the monitored system, then it is enriched with metadata and sent to the centralized platform for processing. Here, two processing options are available. Data can be directly processed with stream-processing patterns, which typically look for indicators of compromise. Data can also be indexed and stored in a database for future retrieval and investigation; in this case, batch processing is able to analyse time series, look for window intervals, make complex correlations between data from different subsystems (i.e., virtual functions). High-level events are produced by both processing branches, which triggers some form of reaction. Reactions may be limited to notification in the user dashboard or may imply the execution of reaction policies by the Security Controller. These latter are specifically designed for specific stream or batch processing programs, because (in a similar way to what happens for other frameworks like Hadoop and Sparks) it is not possible to define common data structures for every possible analytics pipeline, because data depends on the specific application. The design of an analytics pipeline must ensure that the same data structure is used between agents and processing, and between processing and reaction policies.

The common structure is not totally rigid, since some stages might not be present or be implemented as plain passthrough. For example, data enrichment might not be necessary, if the specific agent already provides all necessary information and is able to work as Kafka producer. Additionally, the two processing branches that distil high-level events from raw data are often alternatives. Indexing could also be delegated to default mechanisms of Elasticsearch, in case of very simple data format. And finally, events might be only displayed in the dashboard without triggering any specific action.

The definition of an analytics pipeline requires two elements:

- 1) The description of the pipeline structure, which defines which stages are actually used and the mapping to software modules that implement each of them;
- 2) The implementation of the stages shown in Figure 7, i.e. Raw data extraction, Data enrichment, Stream processing, Indexing, Batch Processing, Reaction; implementation is only required for the stages actually included in the pipeline description, including passthroughs.

It is worth noting that ASTRID follows the micro-services paradigm, hence each software module corresponds to a standalone software unit that interacts through network-based interfaces. This is rather different from other programming models (like Hadoop, Spark) where each function is expected to implement a software interface (e.g., Java interface or C++ class). Hence, the programming model will mainly require programmers to conform to external APIs and interfaces, and so, it does not force the adoption of a specific programming language. The bottom part of Figure 7 shows the primitives used to program the specific component(s) that implement each stage. These include plain changes in the configuration files and management APIs often based on REST paradigms. Kafka, Logstash and Elasticsearch APIs are already well documented within the corresponding projects. The CB API is based on the data model and protocol described in Section 5. A REST-based API is also exposed by the LCP for accessing local agents and Logstash instance; this is not covered in this document, but the live documentation is already available in github⁴.

Even if the interfaces between the different stages are specified (with the only exception of Raw data extraction, which will be discussed later) and most of them require data to be formatted as JSON, data structures are not covered by the programming model. This is fully compliant with the overall objective to provide a platform that facilitate the implementation of multiple security services for virtualized applications, without being limited to pre-defined use cases.

There are different models that must be followed to implement each stage of an analytics pipeline, because different frameworks may be adopted. For this reason, on per stage basis implementation models are independently analysed herein. Furthermore, as one of design objectives is the reuse of available software components (mainly those from the open-source community), these will be referred in the subsection that describes each individual stage implementation.

4.1. Raw Data Extraction

The target is the extraction of a broad range of data, events and measurements from the execution environment of Virtual Functions. Differently from other frameworks, where the number of features is limited by the set of integrated agents, Another target is the definition of “programmable” agents, which are able to run code snippets defined by the security operator. This allows far more flexibility in the definition of new analytics services, without the need to fully re-engineer and re-deploy new agents. The main enabler for this approach is the eBPF technology, which allows to dynamically inject new code in the running kernel, with better guarantees on security and stability than full-fledged kernel modules. When standard agents are used, it is not possible to strictly speaking of “programmability.” In this case, only remote configuration is possible. Indeed, real programmability is possible with DynMon⁵, which provides a common control plane for the injection of eBPF programs and the collection of data they produce.

⁴ <https://astrid-lcp.readthedocs.io/en/latest>.

⁵ <https://polycube-network.readthedocs.io/en/v0.9.0-rc/services/pcn-dynmon/dynmon.html>.

Data plane programming model. A single eBPF program for the TC/XDP hook is required. The programming model for this kind of programs requires an entry function which is called for each packet received and takes as input a reference to the raw packet structure. The program may parse the packet, create statistics, and detect patterns. The output is saved in a specific map, which can be periodically read by the overarching monitoring program.

Control plane programming model. DynMon allows the creation of any table (a.k.a. maps) in the dataplane, which contains the actual raw monitoring data (e.g., IP traffic matrix). Periodically, the content of the table can be read through the Polycube REST interface and sent to the global monitoring system (context broker). To facilitate this operation, a small python program has been created that interact with the DynMon service to inject the eBPF program and read data. Some examples are provided in the DynMon repository⁶.

Input interface. None. Data is generated by agents by parsing log and configuration files, retrieving measurements from the operating system, inspecting packets, and tracing function calls.

Output interface. Unspecified. There is no requirement on the output interface of this module, but it must be matched by the input interface of the next stage. The preferable solution is a plain TCP interface, for which there are already Logstash filter plugins. It is also possible to use HTTP, save data to file, or write directly to the Kafka bus.

Working Example

The following is a working example, taken from the Polycube repository. This JSON file is used to inject the eBPF code by means of the `dynmon_injector.py` program mentioned above; the program counts the number of packets transmitted on the wire and it exports a small table (actually, a one-cell map) with the requested monitoring data.

In short:

- The “code” data represents the actual eBPF code that is injected in the kernel;
- The “metric-configs” section describes the data that is exported by the above program; the most important information is in the “type”: “counter” line, which specifies that the returned data is a single value (a counter, in fact).

```
{
  "ingress-path": {
    "name": "Packets counter probe",
    "code": "\n\n BPF_ARRAY(PKT_COUNTER, uint64_t, 1);\n\n static __always_inline
    int handle_rx(struct CTXTYPE *ctx, struct pkt_metadata *md) {\n\n unsigned int key =
    0;\n\n uint64_t *pkt_counter = PKT_COUNTER.lookup(&key);\n\n if (!pkt_counter){\n\n /*counter map not found !? */\n\n return RX_OK;\n\n }\n\n *pkt_counter+=1;\n\n pcn_log(ctx, LOG_TRACE, \"counter: %d\", *pkt_counter);\n\n return RX_OK;\n\n }",
    "metric-configs": [
      {
        "name": "packets_total",
        "map-name": "PKT_COUNTER",
        "open-metrics-metadata": {
          "help": "This metric represents the number of packets that has traveled through this probe.",
          "type": "counter",

```

⁶ <https://github.com/polycube-network/polycube/tree/master/src/services/pcn-dynmon/examples>.

```

        "labels": []
      }
    ]
  },
  "egress-path": {}
}

```

At run-time, the defined metric can be collected by means of a simple REST call such as the following:

```
HTTP GET http://localhost:9000/polycube/v1/dynmon/monitor/metrics/ingress-metrics/features
```

And the answer would be such as:

```

{
  "name": "features",
  "value": [
    {
      "total_packets": 15
    }
  ]
}

```

4.2. Data Enrichment

Data enrichment is expected to add common metadata (like timestamps, source identifier, geo localization, etc.), make initial filtering and normalization, format the data, and similar operations. It is intended as a sort of convergence layer, where input from heterogeneous data are adapted to the common transport protocol and message format. This stage is implemented by Logstash

Programming model. The creation of code for this layer follows the Logstash model. It is based on three main plugins: input, output, and filter. Input plugins are used to take data from different sources they implement plain TCP/UDP sockets, HTTP servers, file readers, and other protocols. Filter plugin transform and enrich the data, by adding metadata, changing the format of data, structuring data, etc. Output plugins send data through specific protocols: HTTP, Kafka, MQTT, and so on. Logstash programs are developed in Ruby and can be chained to compose processing pipelines. More specific details are available from the Logstash documentation⁷.

Input interface. Unspecified. The input interface must match the output interface of the agent.

Output interface. Data must be written on a given Kafka topic as JSON messages. The data structure is chosen for each specific pipeline. The Kafka plugin has to be used for any pipeline. The Kafka topic should be configurable and set during the pipeline initialization phase.

4.3. Stream Processing

The purpose of this stage is the on-the-fly processing of streams. Typical operations include filtering of relevant events, by looking for potential indicators of compromise, Programmers may need to maintain an internal state while processing data, like counters or more complex structures. The overall model for this stage is to take data from a Kafka topic, process it, and write refined

⁷ Contributing to Logstash. URL: <https://www.elastic.co/guide/en/logstash/master/contributing-to-logstash.html>.

events to another Kafka topic; actually, it can be viewed as a sort of “translator” between two Kafka topics.

There are two optional ways to implement this stage. The first one is writing custom programs that implement both Kafka consumer and producer role. The second one is to leverage the native Kafka streaming framework and makes it accessible as an ordinary application programming model for asynchronous services. In the last case, Java classes must be developed to implement a given interface⁸. Both options can be used in the Kafka platform, but, as the current implementation of Smart Controller has been chosen to be based on a set of custom programs, the definition of the programming model for this stage will focus on this option.

Programming model. A streaming program must take input from a Kafka topic and write output to a different Kafka topic. Both topics should be configurable, so that an analytics pipeline can be created by the Smart Controller. There are no other restrictions or constraints on the implementation of custom streaming programs.

Input interface. A streaming program must consume JSON messages from a given Kafka topic. The structure of the message should be known by the program. This means that the program must be tailored to the output of a specific agent.

Output interface. A streaming program must create JSON messages on a given Kafka topic. Such messages can be visualized in the Dashboard or consumed by the Smart Controller. In the latter case, specific control policies must be developed for each streaming program.

5. Concept and Architecture

ASTRID is a multi-tier architecture, where a common, programmable, and pervasive context fabric feeds a powerful set of multi-vendor detection and analysis algorithms (business logic). On the one hand, the challenge is deep visibility over multiple software components by real-time collection of massive events from a multiplicity of capillary sources, while maintaining essential properties such as forwarding speed, scalability, autonomy, usability, fault tolerance, resistance to compromises, and responsiveness. On the other hand, the ambition is to support better and more reliable situational awareness by inter- and intra-domain data correlation in both space and time, in order to timely detect and respond even the more sophisticated multi-vector and interdisciplinary cyberattacks.

The conceptual structure of the context fabric is depicted in Figure 6. Local agents are deployed in a pervasive and capillary way within the service topology; they represent context sources that gather security related data, measurements, and events from the operating system, daemons, and local applications. The context is collected by the Context Broker (CB), which delivers it to specific processing algorithms for analysis, correlation, and detection; data are also stored to enable historical analysis and forensic investigation. Finally, the CB exposes an abstraction of what can be collected, i.e., the Security Context Abstraction (SCA), which is the concrete implementation of the Security Context Model (SCM).

⁸ Kafka streams documentation. URL: <https://kafka.apache.org/26/documentation/streams>.

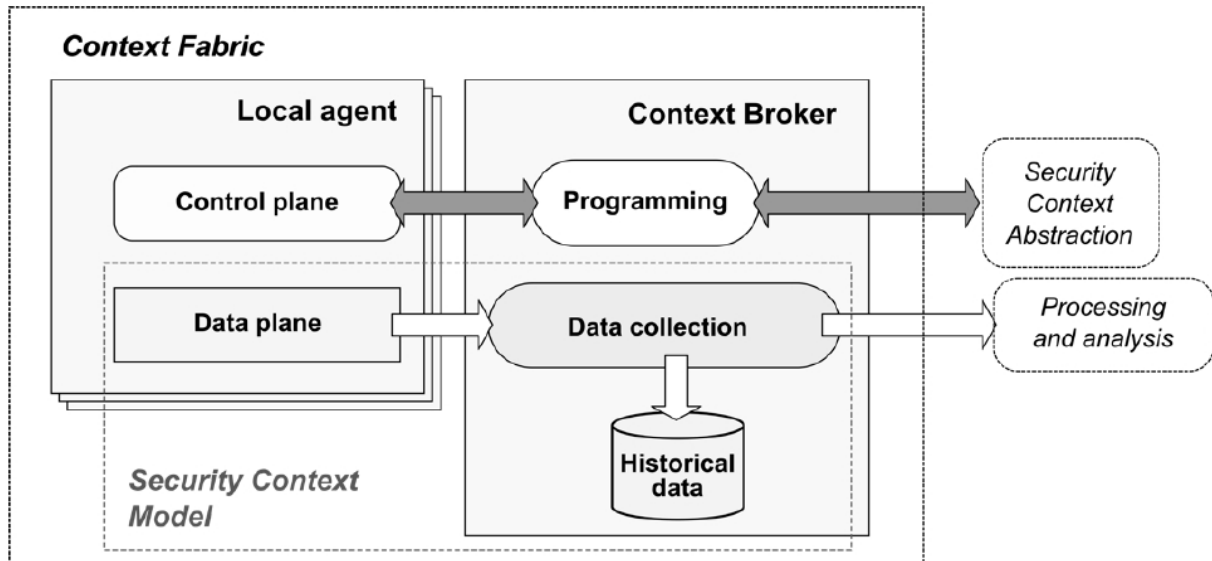


Figure 6. Conceptual structure of the ASTRID context fabric.

5.1. Virtualization Models

The need for ever more agility in the creation, management, and termination of ICT services have been progressively boosted the wide adoption of virtualization technologies. Virtualization enables to decouple the software from the underlying hardware; the main advantages are the possibility to share the same hardware among multiple tenants and quick migration (though under some technical constraints).

Though a systematic dissertation about virtualization is well beyond the scope of this document, a quick overview of the main models and technologies is useful to better understand the scope for the design and implementation of the security context fabric.

Until recently, the most common form of virtualization has been based on hypervisors. A hypervisor is a program that allows multiple operating systems to share the same hardware. By “virtualise”, we mean the division of resources (CPU, RAM etc.) in the physical computing environment (known as a host) into several smaller independent ‘virtual machines’, known as guests. One of the key functions of hypervisors is isolation, meaning that a guest cannot affect the operation of the host or any other guest in any case. This is achieved by hardware emulation of a physical machine and (except under carefully controlled circumstances) prevention of direct access to real hardware by the guests. While in early x86 architecture full virtualization was a cumbersome task, due to the need of binary translation to trap and virtualise the execution of certain sensitivity “non-virtualizable” instructions, the introduction of Intel’s VT and AMD-V extensions in 2005-2006 removed the burden of software emulation for time consuming tasks as ring deprivileging.

The hypervisor layer provides strong isolation between multiple guest operating systems (OSes) to run simultaneously on the same hardware. There are different options for the design and operation of a hypervisor (i.e., tier-1 or tier-2 architecture, full virtualization or para-virtualization), which roughly correspond to the same conceptual layout shown in Figure 7. This model has a large overhead on computing, memory, and storage resources, because a full-blown operating system image runs for each virtual machine. This is acceptable when a single VM is used to run an

additional operating system on the same host, but it might be questionable in case many guest OSes are used to run one or a few applications only. As a matter of fact, lightweight OSes have become common in cloud environments (e.g., CirrOS, Linux Alpine).

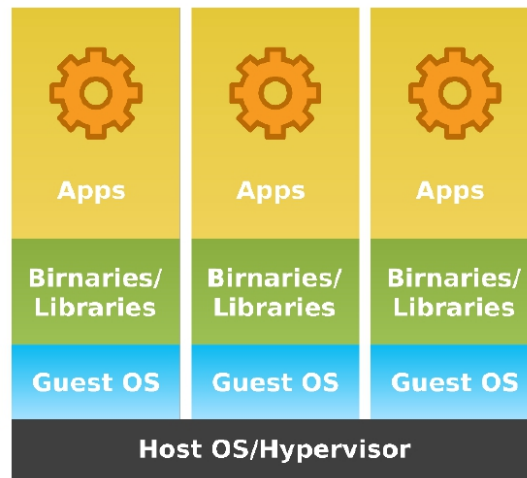


Figure 7. A hypervisor runs multiple isolated Guest OSes concurrently on the same hardware.

More recently, the hunt for efficiency and better performance has raised the interest in alternative technologies based on looser isolation paradigms. Though the ancient “chroot” mechanism cannot fulfil resource and security isolation goals for multi-tenant designs, the introduction of more effective partitioning mechanisms in the Linux kernel paved the road for a new form of virtualization, based on “containers.” Containers allows the existence of multiple isolated user-space instances, each one with its own view of available system resources (connected devices, files and folders, network interfaces, CPU power, quantifiable hardware capabilities); the kernel often provides resource-management features to limit the impact of one container's activities on other containers. The most common containerization technologies are Linux LXC and Docker, but similar solutions are also available in Solaris, BSD, FreeBSD.

The difference between containers and hypervisors is even clearer when comparing their conceptual layouts. Figure 8 shows that all containers share the same kernel, hence reducing the overhead. A deep comparison between the two virtualization technologies should also consider the possibility to migrate to different host, software interdependencies (e.g., failure of the kernel would affect all hosted containers), size and distribution of binary images, resource management, and so on.

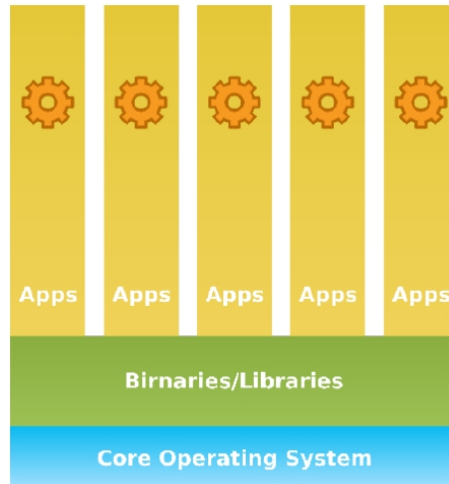


Figure 8. Containers are isolated user-space instances running on the same kernel.

The concept of cloud computing brings virtualization one step further, by abstracting computing, storage, and networking capabilities of large hardware infrastructures. In this case, Cloud Management Software is responsible for management of a large number of hypervisors and/or containers, providing user-friendly interfaces to create VMs or containers, load the software, interconnect them through shared or dedicated (virtual) network links, connect to the Internet, provide virtual consoles, migrate them across the infrastructure, configure firewalling rules, remove them. Example of well-known CMS are OpenStack, VMware vSphere, and Kubernetes. They can be used by humans, or by software orchestration tools.

The ASTRID framework gives visibility over virtual services, independently of the underlying infrastructure and virtualization models. This approach is motivated by the need for portability across heterogeneous systems, so to create a multi-domain tool. In this respect, monitoring and inspection is only considered within each virtualized environment, both in case of VMs and containers. In this document we will refer to execution environment as the isolated partition where the virtual function runs; it could be either a VM or a container, but the difference is not visible to the software hosted inside. It is clear that this difference is evident at the management layer, but software orchestration takes care of this aspect and hides it to ASTRID.

The choice to be infrastructure-agnostic has a clear impact on the threat model and the composition of the security context. As general consideration, ASTRID can monitor all aspects related to software execution within its virtual environment, including network traffic exchanged with external entities; however, ASTRID will not have visibility on the allocation of resources among multiple tenants, network configurations at the infrastructure level (i.e., forwarding rules, VLAN settings, routing topologies, etc.). Following Sections will detail the role of the logical components of the context fabric and the context that will be collected.

5.2. Local agents and programmability

Local agents are the only part of the architecture that are co-located with the virtual service. Their internal structure includes two logical layers: the data plane and the control plane (Figure 6). The data plane collects the security context, i.e., a knowledge base including events (failed login attempts, denied access, system calls), logs (service requests, operations, anomalies, client identity, execution traces, memory dumps), measures (network metrics, usage profiles) that can be useful

for detection of known attacks or identification of new threats. One of the main ambitions for ASTRID is the collection of data from different subsystems (disk, network, memory, I/O), instead of relying on a single source of information as is the common practice nowadays (i.e., in case of flow-monitoring tools, antivirus, intrusion detection systems). Therefore, the data plane implements security hooks for the filesystem, the network, and applications. The data plane would also be responsible for enforcing security policies, including packet filtering, access control, and re-configuration of the execution environment; however, the definition of a complete framework for reaction and mitigation is beyond the scope of ASTRID, and only basic firewalling services will be considered in the Project.

Since the collection of data from multiple sources may easily result in excessive network overhead, it is important to shape the inspection, monitoring, and collection processes to the actual need. One of the main innovation pillars is programmability, i.e. the capability to dynamically adapt operation to the evolving context in both spatial and temporal dimensions, so to effectively balance granularity of information with overhead. The ambition is to go beyond plain configurability, which is a common feature in any modern data plane, targeting the injection of lightweight yet secure code at run-time without affecting operation of the virtual functions. This is important to tackle the continuous evolution of the attack patterns, especially for investigation of and reaction to zero-day attacks, because in this case the set of available configurations might not be able to detect or implement the required features. Programming also includes the capability to offload lightweight aggregation and processing tasks to each virtual environment, hence reducing bandwidth requirements and latency.

The need for programmability entails two main aspects. On the one hand, techniques and mechanisms to program monitoring resources, and this is the main task for the control plane. On the other hand, proper abstraction of the current context, including both the service layout and current measurements, which is a feature of the CB.

In each local agent, the control plane is therefore responsible for programmability, i.e., changing the behaviour of the data plane at run-time. According to the previous considerations, there are two main dimensions for programmability:

configuration@run-time, i.e., the operational parameters are modified according to pre-defined and static templates, patterns, and options. For example: the name of files to be parsed, which should not be limited to log files (e.g., syslog, apache log, authentication and access logs), but should also include configurations (services to run on boot, system users, parameters for applications, etc.); current status (running applications, socket status, on-line users, plaintext/encrypted connections); verbosity of reporting, namely filtering events based on their severity (e.g., debug, info, warning, error); fields in packet headers that can be parsed and compared to specific values. Though some monitoring tools are already designed to be controlled at run-time (especially for networking, where Net-Flow, sFlow, IPFIX, and more recently OpenFlow and NetConf, are widely used for this purpose), in many cases static configuration before running the software is only possible, hence the development of a specific control plane must be taken into account.

code@run-time implies the possibility to run security programs without re-designing, re-deploying, and even re-starting local agents. This feature is challenging both from the technical and security perspective, because the easiness to run scripts or make just-in-time compilation must not turn into a potential threat for the system. In this case, the control plane is the main responsible for verifying authorization, integrity, and safety of any piece of code that is injected

into the data plane.

There is almost no limit to the number of parameters and technologies that could be useful for detection of some threats and therefore would deserve to be included in the security context.

Local agents are the only part of the architecture deployed in the virtual service. As such, their management (installation, configuration, update, removal) is entirely delegated to the same orchestration software used for the virtual service. They should be inserted in the service template during the enrichment phase, i.e., the integration with the ASTRID framework.

The selection of virtual functions that will embed agents and the security hooks present in each agent is part of the security assessment process. The most forward-looking approach would be the deployment of the full set of monitoring capabilities in each virtual function (which are just operated when needed, according to the control policies), but hard constraints on image size or software compatibility might force other designs. Depending on the capabilities of the service orchestrator, local agents might also be deployed and removed at run-time, when requested by security policies or human directives. In any case, the additional management plane for local agents is provided by external software orchestration solutions and will not be considered in this document.

6. Performance Evaluation

This section describes the performance evaluation of the CB-Manager and LCP components. We consider 4 different scenarios in order to evaluate these two components varying the number of the HTTP requests and to verify a correct operation. Table 2 shows a short summary of the tests done. Each scenario requires a series of HTTP requests involving different entities (execution environment, network link, connection, etc.). For this reason, we measured the statistics (average, max, min, standard deviation and median) of the response time (in ms) considering the single HTTP requests and the flow composed of a group of HTTP requests required to complete the scenario. For example, the Service Topology scenario requires 12 different HTTP requests in order to obtain the needed info.

The entire evaluation was done using Apache JMeter⁹. The source code is available in the GitHub¹⁰ repositories of the project. For each scenario we run different tests varying the number of HTTP requests per second from 10 to 100. Hence, considering that each scenario requires from 5 to 10 HTTP requests, the number of requests to complete the scenario is between 1 and 20.

Table 2. Different scenarios for the performance evaluation.

Scenario	Steps	Entity Involved
Service Topology	Get all the data from various endpoints to obtain the info related to the service topology.	ExecEnv
		ExecEnv Type
		Network Link
		Network Link Type

⁹ <https://jmeter.apache.org>.

¹⁰ <https://github.com/astrid-project/astrid-platform/tree/main/test/jmeter>.

		Connection	
		Pipeline	
		Algorithm	Catalog
			Instance
		Agent	Catalog
			Instance
		eBPF-Program	Catalog
			Instance
Execution Environment Setup	Add a new entry in the execution environment and check if the LCP is correctly connected.	ExecEnv	
		ExecEnv Type	
Agent Management (Action)	Add a new agent instance with info from the catalog and execute the start and stop actions.	ExecEnv	
		Agent	Catalog
			Instance
Agent Management (Parameter)	Add a new agent instance with info from the catalog, execute the start action, update a parameter, and finally execute the restart action.	ExecEnv	
		Agent	Catalog
			Instance

6.1. Service Topology

Figure 9 shows the needed HTTP request in order to obtain the related info about the service topology.

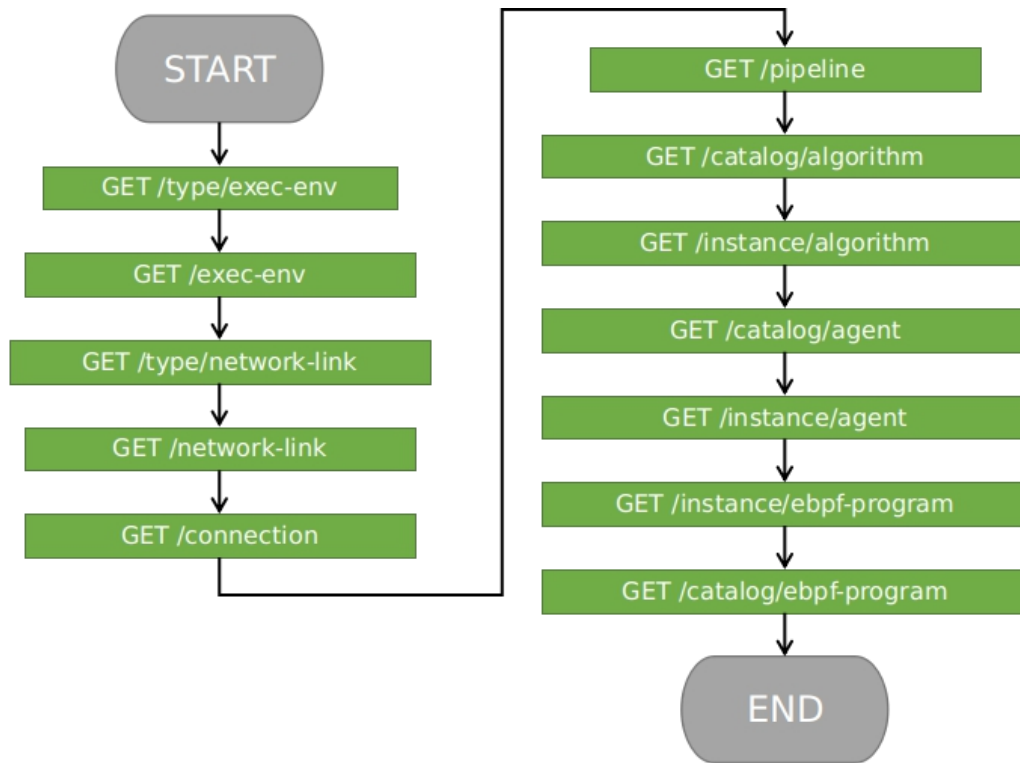


Figure 9. HTTP Request flow of the Service Topology scenario.

The results of the performance evaluation is shown in Table 3 and in Figure 10. Varying the number of HTTP requests per second from 10 to 200, the average response time grows from 295 ms to 500 ms; while the minimum value remains essentially stable. Instead, a slight growth is shown for the maximum value and for the standard deviation one. Finally, the median value follows the same trend of the average case.

Table 3. Performance evaluation of the Service Topology scenario considering the single HTTP requests.

Avg. # HTTP Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
10	295	98	800	206	220
20	296	97	802	210	240
30	295	98	803	211	260
40	298	98	805	225	251
50	320	99	810	232	290
75	370	97	824	250	345
100	400	102	837	300	380
125	410	103	850	320	381
150	415	102	875	340	390

175	435	105	889	302	401
200	501	106	898	301	490

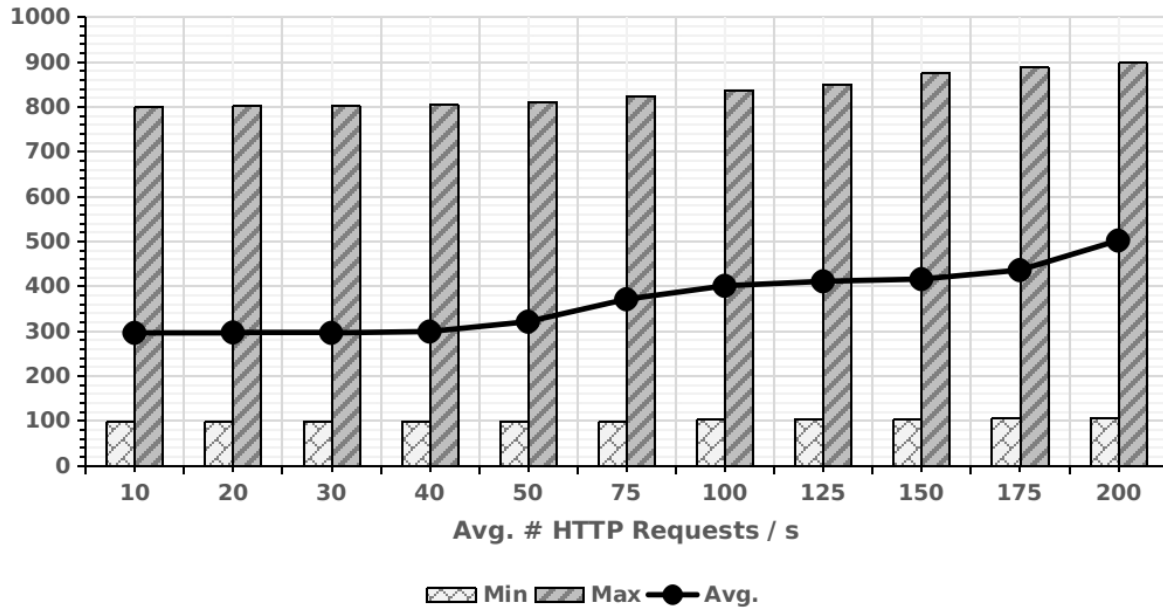


Figure 10. Response Time (in ms) varying the number of the single HTTP requests for the Service Topology scenario.

Instead, the results considering the scenario as a whole are shown in Table 4 and in Figure 11. The trend is similar to the case of single HTTP requests.

Table 4. Performance evaluation of the Service Topology case considering the HTTP requests grouped by scenario.

Avg. # Scenario Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
1	4013	3700	9001	2056	2218
2	4023	3701	9010	2168	2519
3	4017	3703	9021	2162	2504
4	4018	3702	9050	2296	2561
5	4015	3704	9049	2431	2826
7.5	4018	3706	9065	2424	3397
10	4130	3810	9095	2939	3764
12.5	4200	3812	9099	3126	3654
15	4443	3811	9102	3536	3947
17.5	4320	3813	9105	2914	4091
20	5099	3815	9130	2900	5047

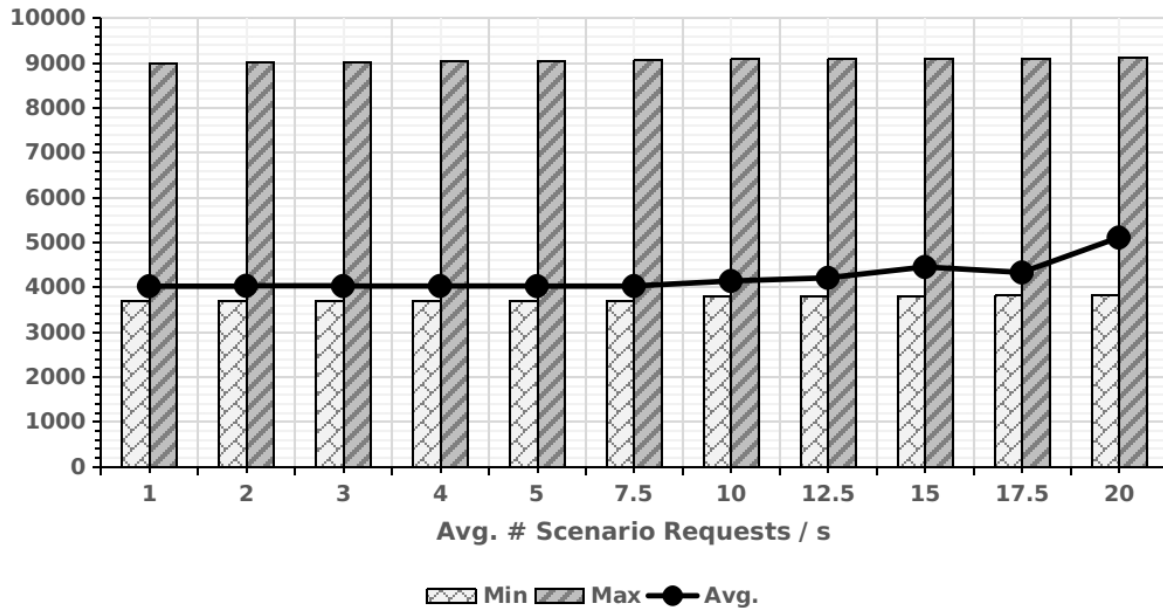


Figure 11. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the Service Topology case.

Finally, Table 5 shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the agents (catalog and instance); while the faster are the ones related to the execution environments (type and exec-env).

Table 5. Performance evaluation of the Service Topology case considering the HTTP requests grouped by type / endpoint with 200 average # HTTP requests.

Request		Response Time [ms]				
Method	End-Point	Avg.	Min	Max	St.Dev.	Median
GET	/type/exec-env	347	106	394	208	339
GET	/exec-env	402	123	457	241	393
GET	/type/network-link	403	123	458	242	394
GET	/network-link	406	124	462	244	397
GET	/connection	489	149	556	293	478
GET	/pipeline	504	154	573	302	493
GET	/catalog/algorithm	568	174	646	341	556
GET	/catalog/agent	723	221	822	434	707
GET	/instance/agent	790	241	898	474	773
GET	/catalog/ebpf-program	489	149	556	293	478
GET	/instance/epf-program	390	119	443	234	381
Total		501	106	898	301	490

6.2. Execution Environment Set Up

With this scenario we test the setup of an execution environment considering the insertion of a new entry and the related synchronization with the LCP running on it. Figure 12 shows the steps needed to complete this scenario.

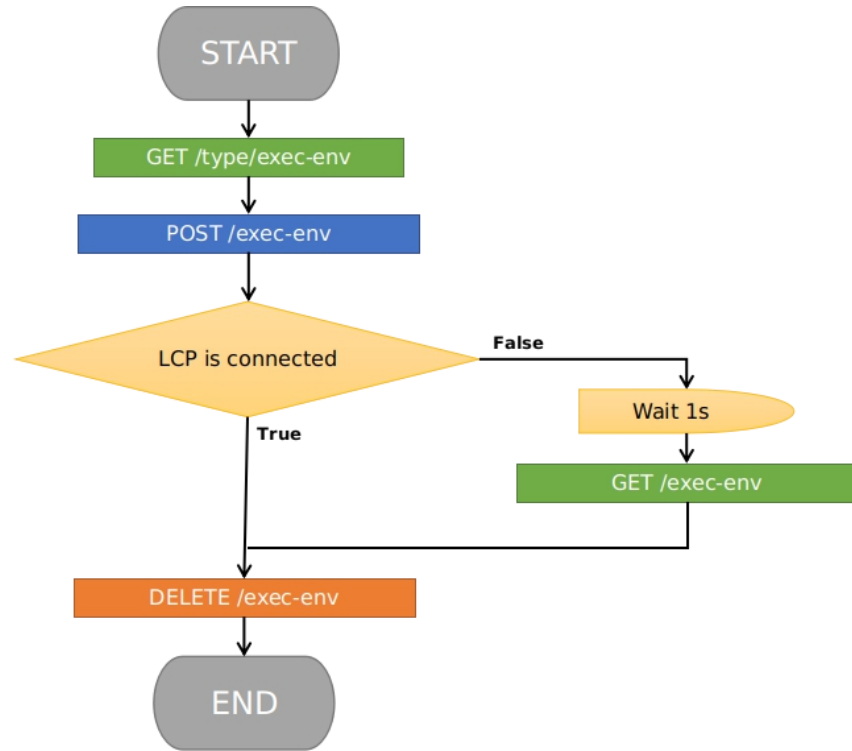


Figure 12. HTTP Request flow of the Execution Environment Setup scenario.

The results of the evaluation are shown in Table 6 and Figure 13. The average response time varies from 300 ms to more than 700 ms. Unlike the previous scenario, there is a slight growth even for the minimum value. The other statistics follow more or less the same trend of the average case with similar consideration.

Table 6. Performance evaluation of the Execution Environment Setup scenario considering the single HTTP requests.

Avg. # HTTP Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
10	309	103	837	215	230
20	379	124	1028	269	308
30	301	100	820	216	266
40	289	95	781	218	243
50	443	137	1122	321	402
75	465	122	1035	314	433

100	561	143	1174	421	533
125	514	129	1065	401	477
150	419	103	884	344	394
175	520	126	1064	361	480
200	719	152	1289	432	703

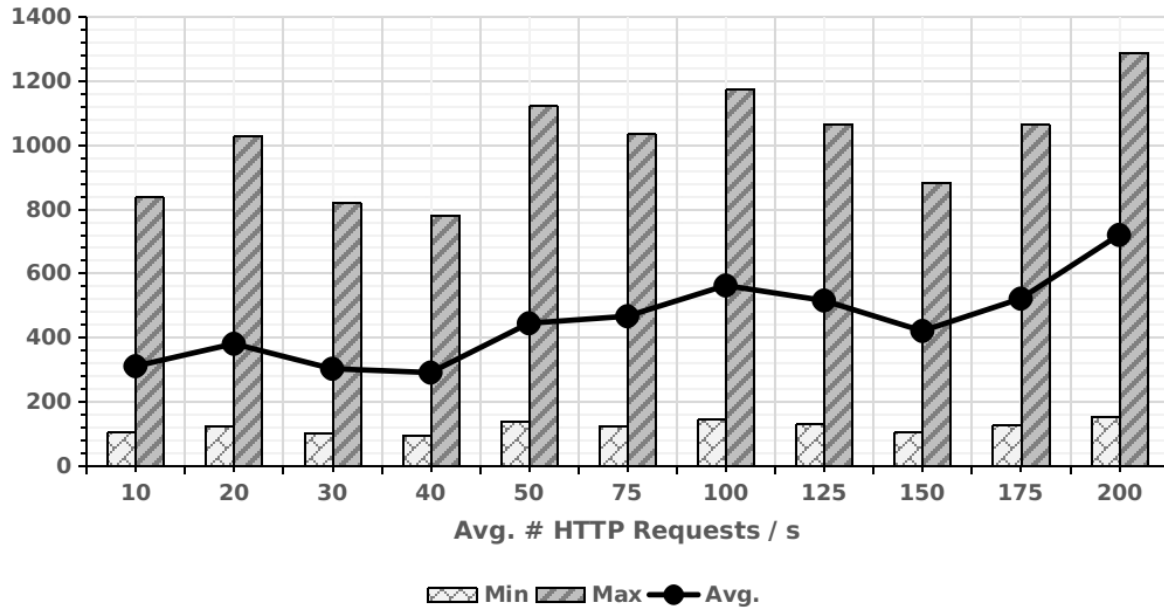


Figure 13. Response Time (in ms) varying the number of the single HTTP requests for the Execution Environment Setup scenario.

Instead, for the results considering the grouped HTTP requests, the difference in the average value as the number of requests increases is less significant (Table 7 and Figure 14). It is even possible to notice a trend that is not totally growing but more undulatory. The other statistics also exhibit these characteristics. Most likely one of the reasons for this variability is due to the synchronization part of the LCP that is managed by the CB-Manager with periodic polling.

Table 7. Performance evaluation of the Execution Environment Setup case considering the HTTP requests grouped by scenario.

Avg. # Scenario Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
1	4900	4518	10991	2511	2708
2	5302	4878	11875	2857	3320
3	4286	3951	9626	2307	2672
4	4163	3836	9377	2379	2654
5	5389	4971	12145	3263	3793

7.5	4817	4443	10869	2906	4073
10	4372	4034	9629	3112	3985
12.5	5276	4788	11430	3927	4590
15	4446	3814	9108	3538	3950
17.5	4469	3945	9420	3015	4232
20	4990	3733	8934	2838	4939

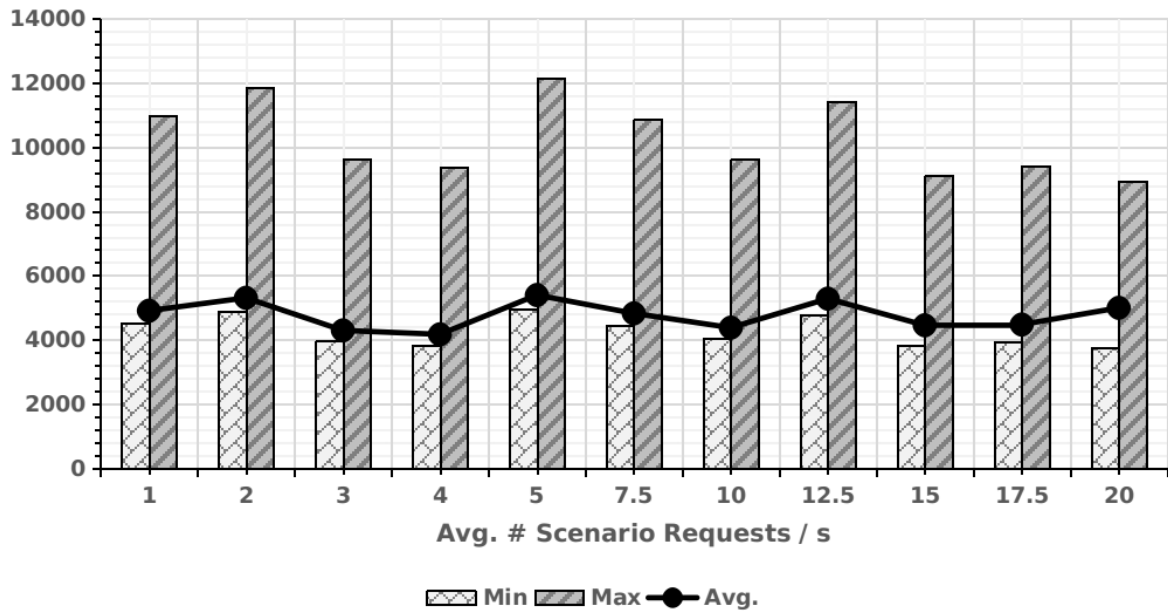


Figure 14. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the Execution Environment Setup case.

Finally, Table 8 shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new execution environment (POST method); while the faster are the ones related to read the info about the execution environment (type and exec-env with GET method).

Table 8. Performance evaluation of the Execution Environment Setup case considering the HTTP requests grouped by type / endpoint with 200 average # HTTP requests.

Request		Response Time [ms]				
Method	End-Point	Avg.	Min	Max	St.Dev.	Median
GET	/type/exec-env	358	152	384	215	350
POST	/exec-env	1201	510	1289	723	1175
GET	/exec-env	425	180	456	256	416
DELETE	/exec-env	890	377	955	536	871
Total		719	152	1289	432	703

6.3. Agent Management

This section includes the two scenarios related to the agent management case. The first one is used to evaluate the execution of the start and stop actions for an agent instance; while the second one to evaluate the update of a parameter of an agent instance.

Action

The steps needed for the action case is shown in Figure 15. The two PUT requests are used to execute the start and stop actions of the agent previously inserted with POST requests (in the catalog and the specific instance deployed in the execution environment).

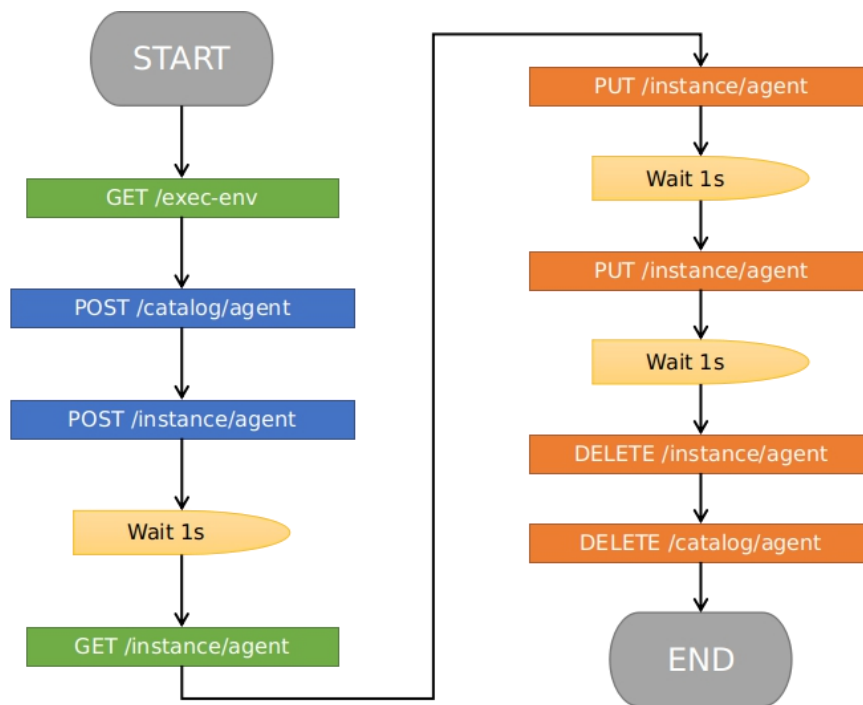


Figure 15. HTTP Request flow of the Agent Management (Action) scenario.

The results are shown in Table 9 and Figure 16. The average response time grows from less of 400 ms to more than 600 ms varying the number of HTTP requests from 10 to 200. Instead, the minimum statistic is more or less constant. Highly variable is the trend of the maximum value. Finally, the standard deviation and the median statistics follow a growing trend, especially for the median one.

As in the previous scenario, there are very fluctuating values due to the execution of the actions by the LCP and the related interaction with the agent running on the execution environment.

Table 9. Performance evaluation of the Agent Management (Action) scenario considering the single HTTP requests.

Avg. # HTTP	Response Time [ms]
-------------	--------------------

Requests / s	Avg.	Min	Max	St.Dev.	Median
10	388	129	1053	271	290
20	387	127	1048	274	314
30	313	104	851	224	275
40	428	141	1156	323	360
50	421	130	1065	305	381
75	436	114	972	295	407
100	546	139	1142	409	519
125	515	129	1068	402	479
150	567	139	1196	465	533
175	446	108	912	310	411
200	617	131	1106	371	604

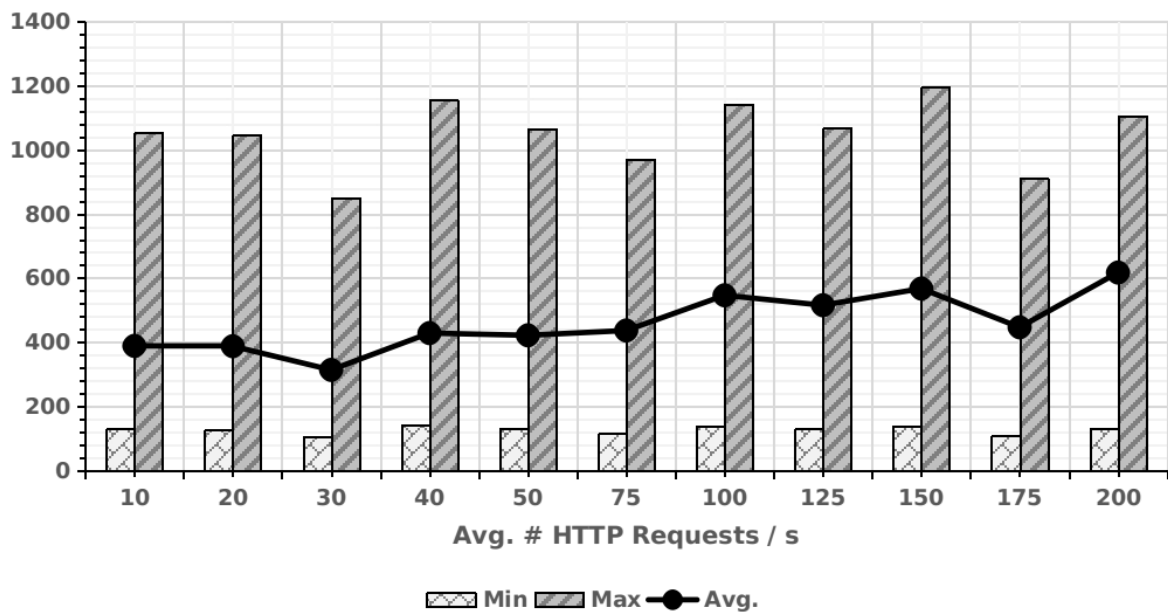


Figure 16. Response Time (in ms) varying the number of the single HTTP requests for the Agent Management (Action) scenario.

The variable values are even more evident when you consider HTTP requests grouped by scenario as shown in Table 10 and Figure 17.

Table 10. Performance evaluation of the Agent Management (Action) case considering the HTTP requests grouped by scenario.

Avg. # Scenario Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
1	3205	2955	7188	1642	1771

2	3926	3612	8792	2116	2458
3	2829	2608	6353	1523	1763
4	2980	2746	6712	1703	1900
5	3943	3638	8888	2388	2776
7.5	2992	2760	6750	1805	2530
10	4118	3799	9068	2930	3753
12.5	2958	2685	6408	2201	2573
15	3190	2736	6535	2539	2834
17.5	3042	2685	6412	2052	2881
20	4572	3421	8187	2600	4526

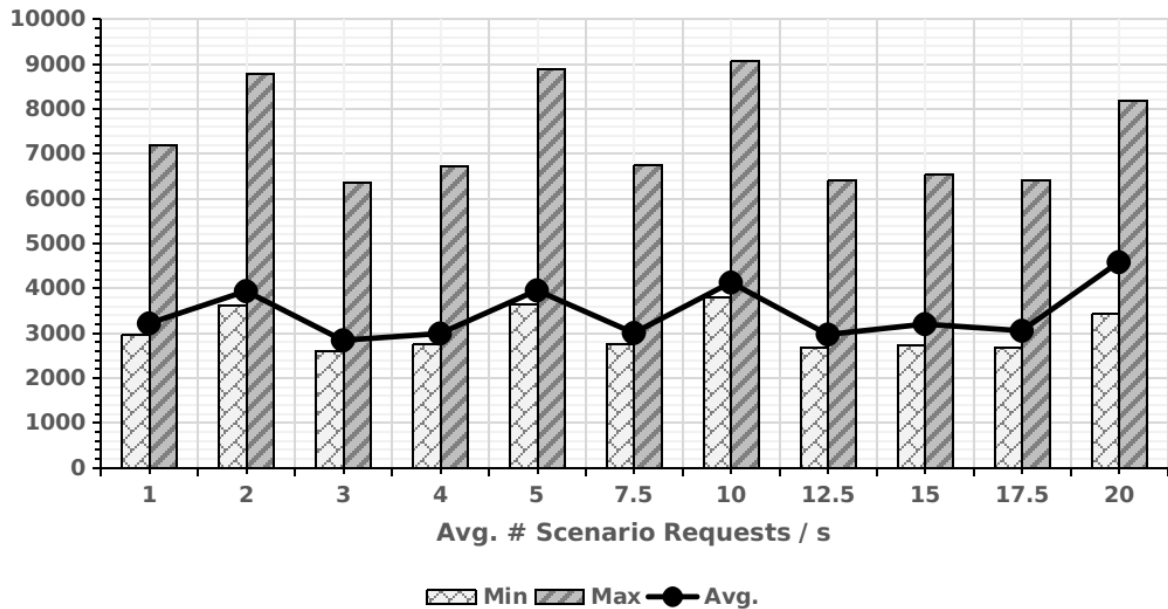


Figure 17. Response Time (in ms) baring the number of the single HTTP requests grouped by scenario for the Agent Management (Action) case.

Finally, Table 11 shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new agent instance (POST method); while the faster are the ones related to read the info about the execution environment (GET method).

Table 11. Performance evaluation of the Agent Management (Action) case considering the HTTP requests grouped by type / endpoint with 200 average # HTTP requests.

Request		Response Time [ms]				
Method	End-Point	Avg.	Min	Max	St.Dev.	Median
GET	/exec-env	201	131	222	121	197

POST	/catalog/agent	654	426	723	393	641
POST	/instance/agent	1001	652	1106	602	981
GET	/instance/agent	423	276	467	254	414
PUT	/instance/agent	696	454	769	418	682
DELETE	/instance/agent	648	422	716	389	635
DELETE	/catalog/agent	694	452	767	417	680
Total		617	131	1106	371	604

Parameter

In the last scenario, we tested the update of a parameter of an agent previously inserted in the catalog and the relative instance. Figure 18 show the steps with the 3 PUT requests needed to start, update the parameter and restart the agent.

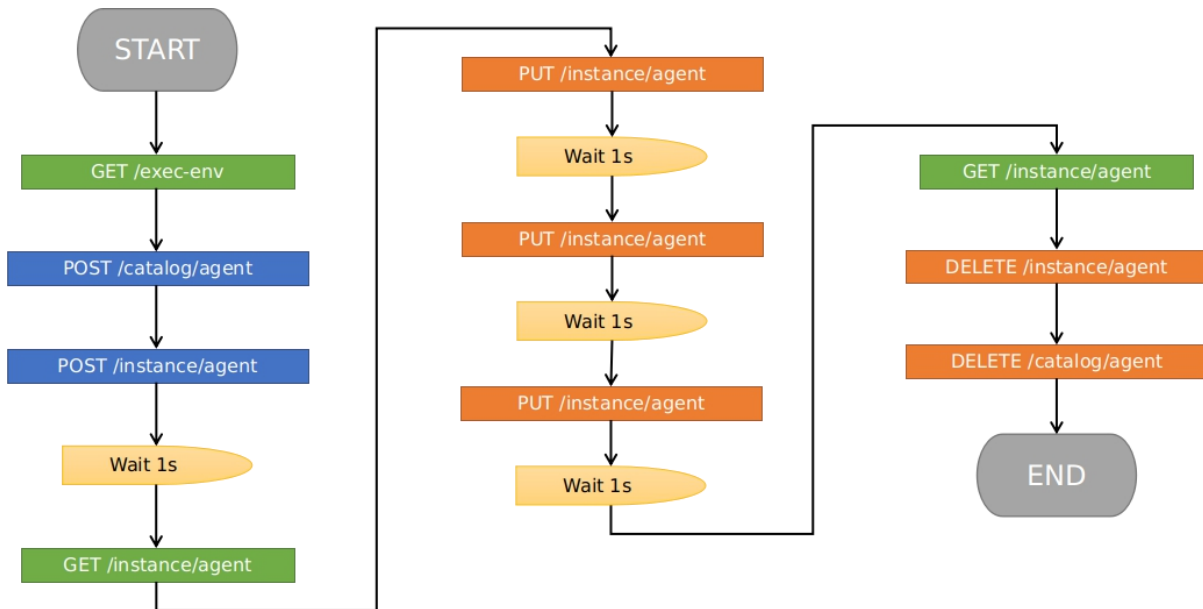


Figure 18. HTTP Request flow of the Agent Management (Parameter) scenario.

The results are shown in Table 12 and Figure 19 show a similar trend of the previous scenario. Also in this case, the high variability is due to the interaction with the LCP and the agent instance deployed in the execution environment.

Table 12. Performance evaluation of the Agent Management (Parameter) scenario considering the single HTTP requests.

Avg. # HTTP Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
10	409	136	1110	286	305
20	428	140	1160	304	347

30	362	120	985	259	319
40	293	96	792	221	247
50	464	143	1173	336	420
75	432	113	962	292	403
100	476	121	996	357	452
125	534	134	1107	417	496
150	561	138	1183	460	527
175	632	153	1292	439	583
200	597	126	1071	359	584

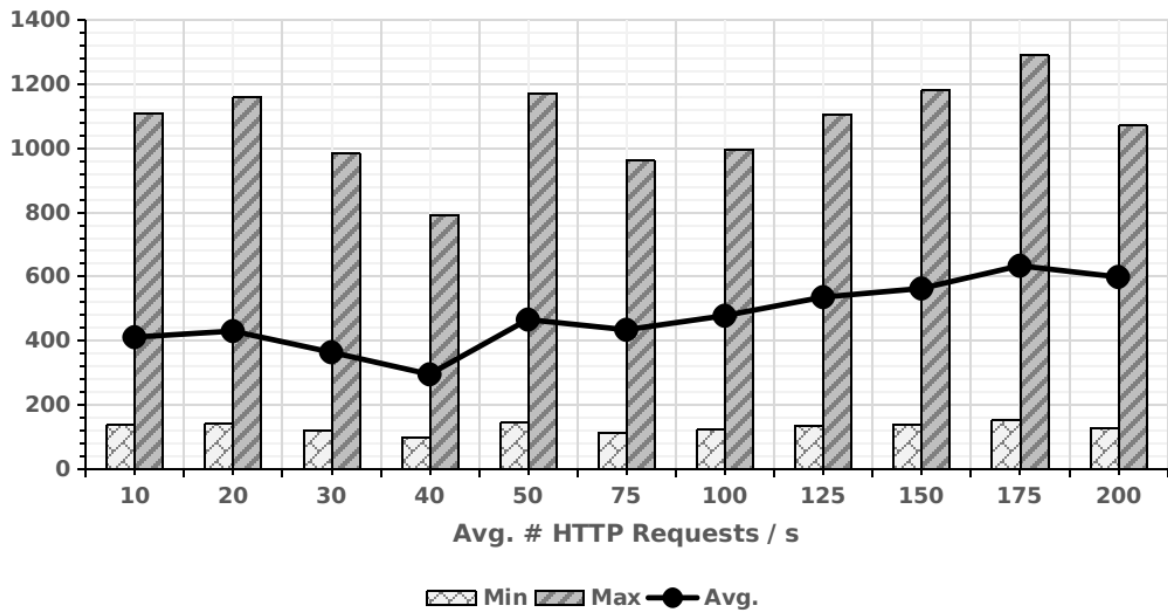


Figure 19. Response Time (in ms) varying the number of the single HTTP requests for the Agent Management (Parameter) scenario.

Finally, similar consideration for the case considering the HTTP requests grouped by scenario as shown in Table 13 and Figure 20.

Table 13. Performance evaluation of the Agent Management (Parameter) case considering the HTTP requests grouped by scenario.

Avg. # Scenario Requests / s	Response Time [ms]				
	Avg.	Min	Max	St.Dev.	Median
1	2942	3165	7699	1759	1897
2	3840	3533	8601	2070	2405
3	3749	3456	8419	2018	2337

4	3699	3408	8332	2114	2358
5	3562	3286	8028	2157	2507
7.5	2895	2670	6532	1747	2448
10	3422	3157	7535	2435	3119
12.5	3053	2771	6615	2273	2656
15	4090	3509	8380	3255	3634
17.5	4095	3615	8631	2762	3878
20	4246	3177	7603	2415	4203

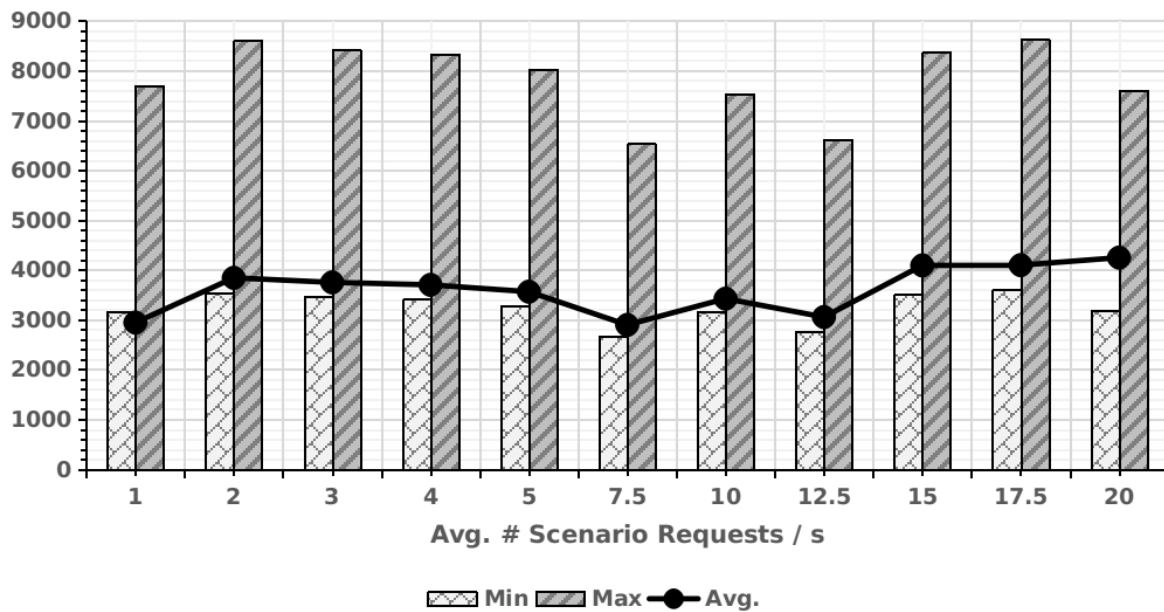


Figure 20. Response Time (in ms) varying the number of the the HTTP requests grouped by scenario for the Agent Management (Parameter) case.

Finally, Table 14 shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new agent instance (POST method); while the faster are the ones related to read the info about the execution environment (GET method).

Table 14. Performance evaluation of the Agent Management (Param) case considering the HTTP requests grouped by type / endpoint with 200 average # HTTP requests.

Request		Response Time [ms]				
Method	End-Point	Avg.	Min	Max	St.Dev.	Median
GET	/exec-env	241	126	266	145	236
POST	/catalog/agent	675	353	745	405	660
POST	/instance/agent	970	507	1071	583	949
GET	/instance/agent	457	239	505	274	447

PUT	<i>/instance/agent</i>	690	361	762	414	675
DELETE	<i>/instance/agent</i>	569	297	628	342	557
DELETE	<i>/catalog/agent</i>	577	302	637	346	565
Total		597	126	1071	359	584

The tests done show a good behavior of the CB-Manager and LCP. The two components are able to satisfy a significant number of requests without loss and with a good response time.